AI Pioneers

CCA-F | Module 3

Lab 3.1

Configuring Claude Code: CLAUDE.md Hierarchy, Commands & Skills

---

Scenario: Setting Up Claude Code for the NorthPeak Pricing Service

Module	M3 — Configuring & Customizing Claude Code
Lab type	Hands-on · Self-paced or Instructor-Led · Run inside Claude Code (VS Code)
Duration	~40 minutes (hands-on)
Environment	Blue Labs VM · VS Code + Claude Code extension · Python 3.10+ · pytest
Sections	S1 (CLAUDE.md Configuration Hierarchy & Modularity) S2 (Custom Slash Commands & Skills)

Lab Objectives

By the end of this lab you will be able to:

- **Structure project memory with CLAUDE.md and @import** — keep rules modular in `.claude/rules/`, wire them into `CLAUDE.md`, and understand the hierarchy that layers user-level memory over project memory.
- **Create slash commands** — turn recurring chores into `/test` and `/review` by adding Markdown files to `.claude/commands/`, with frontmatter for `description`, `allowed-tools`, and `argument-hint`.
- **Package a skill** — write a `SKILL.md` whose `description` lets Claude auto-invoke a repeatable workflow (here, writing a changelog entry).

1. Overview

Out of the box, Claude Code knows nothing about your team's conventions. This lab configures it three complementary ways so good practice becomes the default. First you give it project memory: a short CLAUDE.md that @imports small, modular rule files, plus a user-level rule that layers on top. Then you turn the team's recurring chores into one-word

slash commands. Finally you package a multi-step workflow as a skill that Claude reaches for on its own. Everything is done inside Claude Code, run from the project folder, on the NorthPeak Outfitters pricing service.

SN	Section	What you will do	Core idea
Ex 1	**S1 — CLAUDE.md & @import**	Read CLAUDE.md, see how it @imports rule modules, and add a user-level rule.	Modular memory, layered user + project + imports.
Ex 2	**S2 — Slash Commands**	Inspect and run /test and /review; edit the review checklist.	Recurring chores become shared, one-word actions.
Ex 3	**S2 — Skills**	Inspect the changelog SKILL.md and trigger it by description.	Package a workflow Claude auto-invokes on intent.

2. Scenario

You are setting up Claude Code for the NorthPeak Outfitters pricing service — a small Python library that computes member discounts, shipping, and order totals. The team wants three things: Claude should already know their style and testing rules without anyone pasting them; common chores (run the tests, review a change) should be one word; and turning a change into a changelog entry should be a packaged, repeatable workflow rather than something each person re-explains.

The repo already contains the code, a green pytest suite, and a starter .claude/ config. Your job is to understand each piece of configuration, exercise it, and make it your own.

You will:

- Read CLAUDE.md, follow its @import lines to the modular rule files, and add a user-level rule that layers on top of the project rules.
- Inspect the /test and /review command files, run them, and edit the review checklist to fit the team.
- Open the changelog skill, then trigger it by asking for a changelog entry in plain language.

The configuration at a glance

```
CLAUDE.md            →  project memory; @imports the rule modules below
.claude/rules/       →  style.md, testing.md   (imported into memory)
.claude/commands/    →  test.md → /test,  review.md → /review
.claude/skills/      →  changelog/SKILL.md   (auto-invoked by description)
```

Why these three together?

Memory, commands, and skills capture the team's knowledge in the repo at three grains. Memory sets the standing rules the agent always follows; commands give deliberate, one-word actions you invoke on demand; skills package

multi-step workflows the agent reaches for when intent matches. Together they make good practice the default — consistent across people, versioned in git, and improvable by pull request — instead of living in each person's head.

3. Pre-requisites

3.1 Skilljar Videos — Complete Before the Session

Course	Lessons to watch	Covers
Claude Code 101	The CLAUDE.md file · Skills	**S1, S2**
Claude Code in Action	Custom commands	**S2**
Introduction to Agent Skills	Creating your first skill · Configuration and multi-file skills · Sharing skills (supporting)	**S2**

The lab assumes you have completed the Module 2 labs, so you are comfortable starting Claude Code and using its built-in tools. The instructor will not re-teach Claude Code basics — bring questions to the Doubt Session instead.

3.2 Environment Check

Run these checks before the session starts. If any step fails, contact the instructor or check the Blue Labs SOP.

- Start your Blue Labs VM: open https://www.bluelabs.studio/, locate your VM, click Start, then Connect.
- Get the lab bundle file (lab-3.1-claude-config.zip) from your instructor or your Blue Labs VM, then unzip it and open the folder in VS Code (File > Open Folder...):

```
unzip lab-3.1-claude-config.zip
code lab-3.1-claude-config
```

- Install the Claude Code extension (publisher: Anthropic) from the VS Code Marketplace.
- Create a Python environment and install the test dependency:

```
python -m venv .venv && source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install -r requirements.txt          # pytest>=8.0.0
```

- Confirm the suite is green, then start Claude Code from this folder:

```
pytest -q          # expect: 4 passed
claude             # or use the Claude Code extension panel
```

Where Claude Code loads config from

Start Claude Code from the project root so it finds this folder's `CLAUDE.md` and `.claude/` directory. Project memory and user-level memory (`~/.claude/CLAUDE.md`) are loaded automatically; commands and skills are discovered from `.claude/commands/` and `.claude/skills/`. If something doesn't load, run `/help` and check

the docs — config formats can change between versions. On first launch, Claude Code prompts you to sign in — follow the prompt. The bundle is already a git repository with a committed baseline, so /review and the changelog skill can see your edits via git diff (no git init needed).

4. Exercises

Do these inside Claude Code, started from the project folder. Each is short. The prompts below are what to type at Claude Code; quoted lines in monospace are example questions to ask.

Exercise 1: CLAUDE.md hierarchy and @import (S1) ~12 min

Background

CLAUDE.md is project memory Claude Code reads automatically when you work in the folder. Rather than one giant file, this team keeps small rule files in `.claude/rules/` and pulls them in with `@import`. Claude Code merges memory from several levels — user-level `~/.claude/CLAUDE.md` (every project), the project `CLAUDE.md`, and any `@import`ed modules — with the more specific level taking precedence on conflicts.

Open `CLAUDE.md` and note that it wires in the rules instead of listing them:

```
@.claude/rules/style.md
@.claude/rules/testing.md
```

Task

Confirm Claude already knows the imported rules, then add a user-level rule and watch it layer on top of the project rules.

Step 1 — Read the root memory and its imports

Open `CLAUDE.md`, then open the two files it imports, `.claude/rules/style.md` and `.claude/rules/testing.md`. Notice the root file stays short — it is a table of contents that wires modules together.

Step 2 — Ask Claude about a rule it never had to open

Ask a question whose answer lives only in an imported file:

```
What are this project's testing rules?
```

Expected: Claude answers from `testing.md` (tests for every behaviour change, sentence-style names, cover the boundary and both sides, `pytest -q` must pass) without you opening the file — because memory was loaded at startup.

Step 3 — Add a user-level rule and see the layering

Create your user-level memory `~/.claude/CLAUDE.md` (it applies to every project) with one rule:

```
- Always explain a change in one sentence before editing files.
```

Because user-level memory loads only when Claude Code starts, restart your Claude Code session after creating this file so the new rule is picked up. Then ask Claude to make a small edit, e.g. "add a `loyalty_points` helper to src/northpeak/pricing.py." Watch it follow all three levels: explain first (user rule), keep the function pure and validate

inputs (style.md), and add a test (testing.md). This edit only demonstrates the layering, so discard it before Exercise 2 — in the VS Code Source Control panel choose Discard All Changes (or run git restore . && git clean -fd src/) — so the test suite returns to its original 4 tests.

What good looks like

CLAUDE.md stays short and @imports style.md and testing.md; the testing-rules question is answered from the imported file without opening it; and the user-level rule visibly layers on top — Claude explains before editing (user) while still adding a test (project).

Reflection Questions

- Why keep rules in small @imported files instead of pasting everything into one big CLAUDE.md — what do you gain in maintenance and reuse?
- Claude answered the testing-rules question without opening testing.md. What does that tell you about when project memory is loaded, and why does that matter for every later request?
- A user-level `~/.claude/CLAUDE.md` rule and a project rule could conflict. How does the hierarchy resolve that, and when would you put a rule at the user level vs. the project level?

Exercise 2: Slash commands (S2) ~15 min

Background

A custom slash command is just a Markdown file in `.claude/commands/` — the filename is the command name (so `test.md` becomes `/test`). YAML frontmatter configures it: `description` (shown in the menu), `allowed-tools` (the only tools it may use), and `argument-hint`. The token `$ARGUMENTS` injects whatever the user typed after the command.

Task

Inspect both command files, run them, and edit the review checklist so the command fits your team.

Step 1 — Read the command files

Open `.claude/commands/test.md` and `.claude/commands/review.md`. In `review.md`, note the read-only `allowed-tools` and the `$ARGUMENTS` placeholder — the command can look but not edit.

Step 2 — Run /test

Run the test command and read its summary:

```
/test
```

Expected: Claude runs `python -m pytest -q` and reports `4 passed`. (If something failed, the command would name the failing tests and explain the likely cause — without changing code.)

Step 3 — Make a change, then run /review

Make a small change to `src/northpeak/pricing.py` — for example add a new public function `gift_wrap_fee` without a test — then run:

```
/review pricing.py
```

Expected: Claude runs `git diff`, applies the four-point checklist, and groups findings as blocker / suggestion / nit — flagging the missing test (testing.md) and the missing type hint/docstring (style.md) — ending with a verdict like `Needs changes: 3`. The `pricing.py` argument arrives as `$ARGUMENTS`.

Step 4 — Make the command your own

Open `review.md` and edit the checklist (add a rule, reword a check). Re-run `/review` and confirm your change takes effect. Because the file is in the repo, your whole team gets the updated checklist.

What good looks like

/test reports the pass/fail count from a real pytest run; /review reads the diff and returns findings grouped blocker/suggestion/nit ending in a one-line verdict, and never edits files (its allowed-tools are read-only). The command name matches the filename exactly.

Reflection Questions

- The /review command lists read-only `allowed-tools` and says “do not edit files.” Why scope a command's tools so tightly — what does least privilege buy you here?
- /test and /review are just Markdown files in the repo. What do you gain by checking them in versus each person typing the prompt by hand every time?
- What is `$ARGUMENTS` for in `review.md`, and how does it let one command serve many situations?

Exercise 3: Skills (S2) ~12 min

Background

A skill is a folder with a `SKILL.md` whose frontmatter `description` tells Claude *when* to use it. Unlike a slash command, a skill is **auto-invoked** — Claude reaches for it when a request matches the description, so you never call it by name. This repo ships a `changelog-entry` skill that turns code changes into a Keep-a-Changelog entry.

Task

Read the skill, then trigger it with a plain-language request and check the entry it writes.

Step 1 — Read the SKILL.md

Open `.claude/skills/changelog/SKILL.md`. Note the frontmatter `name` and `description` (the trigger), then the numbered steps and the output format. The description names concrete triggers like “update the changelog” and “release notes.”

Step 2 — Make a change to summarize

Make (or keep) a small change — e.g. the `gift_wrap_fee` helper from Exercise 2, or a one-line fix. The skill summarizes what changed, so it needs something in the diff.

Step 3 — Trigger the skill by description

Ask in plain language — do not name the skill:

```
Update the changelog for this change.
```

Expected: Claude recognizes the request matches the skill's description, follows its steps (look at `git diff`, group under Added/Changed/Fixed/Removed, write user-facing sentences), and prepends a `## [Unreleased]` entry to `CHANGELOG.md` (creating the file if needed).

A correct entry looks like:

```
## [Unreleased]

### Added
- Optional gift-wrap fee helper for orders.
```

What good looks like

Asking “update the changelog” triggers the skill by description (you never name it); the entry groups changes under Added/Changed/Fixed/Removed, each a short user-facing sentence; and it is prepended under ## [Unreleased] in CHANGELOG.md.

Reflection Questions

- A skill is auto-invoked by its description; a slash command is called explicitly by name. When is each the right way to package a piece of work?
- Why does the quality of the `description` field matter so much for a skill — what happens if it is too narrow, or too broad?
- The SKILL.md bakes in judgment (“user-facing sentences,” “skip formatting-only edits”). Why encode that in the skill rather than leaving it to each run — how does this connect to why rules live in CLAUDE.md?

5. Debrief & Reflection

5.1 Self-Check Before You Leave

Answer each question without looking back.

#	Question
1	Where does project memory live, how does @import work, and what are the three levels of the memory hierarchy?
2	How is a slash command defined, and what determines its name? Name the three frontmatter fields used in this lab.
3	Why scope a command's allowed-tools, and what does $ARGUMENTS do?
4	What makes a skill fire? How is that different from invoking a slash command?
5	Across all three, what is the common payoff of putting configuration in the repo?

5.2 Common Mistakes

Mistake	Why it matters and what to do instead
Pasting all rules into one huge CLAUDE.md.	Hard to maintain and easy to ignore. Keep rules small in .claude/rules/ and @import them; edit the module, not the index.
Starting Claude Code from the wrong folder.	If you don't start from the project root, it won't find this CLAUDE.md or .claude/. Open the folder, then start Claude Code there.
Giving a command broad tools "just in case."	A review command with edit tools can change code while "reviewing." Scope allowed-tools to exactly what the command needs.
Naming a command file to mismatch the command.	The filename IS the command name. review.md is /review; renaming the file renames the command.
Vague skill description.	If the description doesn't name real triggers, the skill never fires (or fires wrongly). Describe concrete phrases that should invoke it.

6. Quick Reference

6.1 CLAUDE.md & @import

```
# CLAUDE.md  (project memory, loaded automatically)
## How this file is organized
@.claude/rules/style.md      # inline another file here
@.claude/rules/testing.md

# Hierarchy (more specific wins on conflict):
#   ~/.claude/CLAUDE.md   user level  (every project)
#   ./CLAUDE.md           project level
#   @.claude/rules/*      imported modules
```

6.2 Slash Command Anatomy

```
# file: .claude/commands/review.md   ->   command: /review
---
description: Review current changes against the checklist
allowed-tools: Bash(git diff:*), Bash(git status:*), Read, Grep   # least privilege
argument-hint: "[optional path or scope]"
---
Review the uncommitted changes (focus: $ARGUMENTS) ...
```

6.3 Skill Anatomy

```
# file: .claude/skills/changelog/SKILL.md
---
name: changelog-entry
description: >
  Use when the user wants to add a CHANGELOG entry, write release
```

```
  notes, or summarize a change.   # this is the trigger
---
# Steps ...  (auto-invoked when a request matches the description)
```

6.4 Config Locations & Precedence

What	Where	How it's used
User memory	`~/.claude/CLAUDE.md`	Loaded for every project (lowest precedence).
Project memory	`./CLAUDE.md`	Loaded for this repo; can @import modules.
Rule modules	`.claude/rules/*.md`	Inlined into memory via @import.
Slash command	`.claude/commands/<name>.md`	Invoked explicitly as /<name>.
Skill	`.claude/skills/<n>/SKILL.md`	Auto-invoked when a request matches its description.

6.5 File Inventory

File	Section	Purpose
`CLAUDE.md`	**S1**	Root project memory; @imports the rule modules.
`.claude/rules/style.md`, `testing.md`	**S1**	Modular style and testing rules, imported into memory.
`.claude/commands/test.md`, `review.md`	**S2**	The /test and /review slash commands.
`.claude/skills/changelog/SKILL.md`	**S2**	The changelog-entry skill (auto-invoked by description).
`src/northpeak/`, `src/tests/`	**all**	The pricing code and its green pytest suite.

6.6 Further Reading

- Claude Code — memory & CLAUDE.md: https://docs.claude.com/en/docs/claude-code/memory
- Claude Code — slash commands: https://docs.claude.com/en/docs/claude-code/slash-commands
- Claude Code — skills: https://docs.claude.com/en/docs/claude-code/skills
- Keep a Changelog (format used by the changelog skill): https://keepachangelog.com